

Information Retrieval Assignment-1(CSD358)

MM:20

Deadline: September 12, 2026(11:59 pm)

Note: Do the assignment in group of two students.

Build a small clothing search engine using a corpus of 100 product descriptions.

Learning Objectives

- Build an inverted index with term frequencies and document frequencies.
- Implement ranked retrieval using the Vector Space Model (VSM).
- Compute Inc.ltc cosine similarity using log-tf and idf weighting.
- Extend the inverted index to a positional index.
- Use term positions to support phrase and proximity-style searching.
- Compare retrieval results before and after positional indexing.

Part A — Corpus and Pre-processing

Use the supplied 100-document clothing corpus. Each document represents a clothing product description such as a T-shirt, shirt, jeans, kurta, saree, dress, hoodie, jacket, leggings, or sweatshirt.

- Read the document ID, title/category and description.
- Tokenize the text and normalize case.
- Remove punctuation and apply the stemming method.
- Document and justify your stop-word policy. You may use a standard English stop-word list, but apply it consistently.
- Build a dictionary containing at least term, document frequency (df), and postings.

Part B — Vector Space Model

Implement ranked retrieval for free-text queries. For each term, the postings list should store at least (docID, term frequency). Also maintain document-length information for cosine normalization.

Use the Inc.ltc weighting scheme:

- Document weight: $wd,t = 1 + \log_{10}(tf)$ for $tf > 0$; no idf is applied to document weights.
- Query weight: $wq,t = (1 + \log_{10}(tf)) \times \log_{10}(N/df)$, where $N = 100$.
- Normalize document and query vectors.
- Compute cosine similarity between the query vector and each matching document vector.
- Return up to 10 document IDs, sorted by decreasing similarity; break ties by increasing document ID.

Part C — Positional Index

Extend your inverted index so that every posting stores the positions at which the term occurs in the document. A suitable conceptual representation is:

$(\text{term} \rightarrow \text{df} \rightarrow [(\text{docID}, \text{tf}, [\text{p1}, \text{p2}, \dots]), \dots])$

- Store positions after the same tokenization/normalization/stemming pipeline used in Part A.
- Implement exact phrase search, for example: 'cotton shirt' or 'stretch denim'.
- Implement an ordered proximity search in which two query terms must occur within k token positions.
- Clearly understand the difference between ordinary VSM retrieval and positional retrieval.
- For a phrase query, use positions rather than merely checking whether both terms occur in the same document.

Part D — Application Requirements

- Implement a simple interface where a user enters a free-text clothing query.
- Display the top 10 ranked results with docID, product title/category, and cosine score.
- Provide a second search mode for phrase/proximity queries using the positional index.
- Show the matching term positions for at least one positional query as evidence that the positional index is being used.

Part E — Testing

Do not hard-code expected document IDs.

Mandatory tests:

- Run at least 10 free-text queries.
- Run at least 5 exact phrase queries.
- Run at least 3 proximity queries with different values of k.
- Include at least one query containing a term that does not occur in the corpus.
- Report the top-10 results and explain at least two cases where positional information changes the result set/order.

Suggested Phrase / Positional Queries

- "cotton shirt"
- "stretch denim"
- "festive wear"
- "winter wear"
- "regular fit"
- "breathable fabric"
- "zip closure"
- "high waist"

Sample Proximity Queries

- cotton WITHIN/3 shirt
- stretch WITHIN/4 denim
- winter WITHIN/3 wear
- festive WITHIN/4 kurta

Deliverables

- Source code with comments/documentation.
- Dictionary/inverted-index output.
- Positional-index output.
- Screenshots of the application and representative query results.
- A single ZIP file containing all required files.